

Othmane Zizi

Detailed speaker notes — predictive modelling

Slides 7–11 | target $\approx 2:00$ (stretch 2:30)

Your run of slides

7. Modelling approach — multi-model search
8. MLflow tracking + registry
9. Results
10. Explainability (SHAP + LIME)
11. Fairness + four-type drift

Numbers cheat-sheet

- Mean target $\approx \mathbf{0.33}$ trips / 15-min slot
- Tuned val RMSE **0.994** dep / **0.976** arr
- Naive val RMSE 1.040 dep / 1.027 arr ($\approx 4\%$ **better**)
- Test R^2 **0.334** dep / **0.339** arr
- R^2 by tier (dep, test): **0.018** / **0.066** / **0.394**
- MLflow runs: **73** departure / **57** arrival

Hand-off in (from Louis): “Thanks, Louis. So we have clean, leakage-safe feature tables and a pipeline to train on. I own the modelling — how we pick a model, track it, and prove it’s both accurate and responsible.”

Slide 7 — Modelling: a multi-model search, auto-selected

($\approx 25\text{--}30s$)

“We deliberately did *not* hand-pick a model. We run a search. We start with two strong gradient-boosting baselines — LightGBM and XGBoost. On top of that we add **FLAML**, an AutoML layer that searches model families and configurations, and **Optuna**, which does Bayesian hyperparameter tuning. *Every* trial is logged to MLflow. The pipeline then selects the best model purely by **validation RMSE** and promotes it automatically. We run this once per target — departures and arrivals — and, independently, both land on the same winner: an Optuna-tuned LightGBM, ‘lgbm_optuna’. So the choice is automatic and reproducible, not a guess.”

Hit these:

- Candidates (LightGBM + XGBoost) + **FLAML AutoML** + **Optuna HPO**; every trial $\rightarrow$ MLflow.
- Selection is by **validation RMSE**, automatic; both targets independently pick lgbm_optuna.

Transition: “Because every trial is tracked, we can stand behind that choice — here’s the tracking.”

Slide 8 — MLflow tracking + registry

($\approx 20\text{--}25s$)

“All of that is fully traceable. We run an MLflow tracking server on EC2 with an S3 artifact store. The departure experiment has **73 runs**, arrival **57** — every candidate, FLAML and Optuna trial, with its parameters, metrics and the model itself. The single best run per target is registered in the model registry and promoted to the **production** alias — and that production model is exactly

the artifact our apps load and serve. No retraining drift between what we evaluated and what we ship.”

Hit these:

- Tracking server on **EC2 + S3**; experiments `bixi-demand-departure / -arrival`.
- Best run → **registered** → **production** alias = the exact artifact served.

Transition: “So how good is that production model? Here are the numbers.”

Slide 9 — Results

(*≈30–35s — your anchor slide*)

“Two things to read here. First, be honest about the metric: 15-minute, per-station demand is genuinely noisy — the average slot is about a third of a trip, and most slots are zero — so absolute R^2 is modest *by design*. What matters operationally is the error size, and that’s well under **one trip per slot**: RMSE around 1.0, typical error about 0.6.

Second, it’s a real model, not a baseline. Against a naive historical-average baseline we cut validation RMSE by about **4%** and add roughly six points of R^2 . It’s **stable across two unseen months** — May and October 2025 — so no overfitting. And the predictive power is concentrated exactly where decisions are made: on high-demand stations the R^2 is **0.39**, versus near zero on the quietest ones.”

Hit these:

- Errors < **1 trip / slot** (RMSE $\approx$ 1.0, MAE $\approx$ 0.6); mean target only $\approx$ 0.33.
- **Beats naive** ($\approx$ 4% RMSE, +6 pts R^2); **stable** on two unseen months.
- R^2 by tier (dep, test): low 0.018, medium 0.066, **high 0.394** — strong where it counts.

Transition: “And we can explain why it predicts what it does.”

Slide 10 — Explainability: SHAP & LIME

(*≈20–25s*)

“We explain both models with SHAP and LIME. The SHAP beeswarms tell a clean, sensible story: most of the signal comes from the **2024 historical baselines** — recent-lag and historical-average demand — and from the **cyclical time-of-day** features. Weather is a real but *secondary* driver. It’s the same picture for departures and arrivals, which is reassuring — the model is leaning on stable demand structure, not noise. LIME lets us explain any single prediction on demand, which matters for operator trust.”

Hit these:

- Top drivers: `baseline_prev_15min`, `hist_avg_demand`, cyclical time; weather secondary.
- Same story dep & arr → sensible behaviour; **LIME** for per-prediction explanations.

Transition: “Explainability leads straight into responsible AI — fairness and drift.”

Slide 11 — Fairness & four-type drift

(*≈25–30s*)

“Two responsible-AI checks. **Fairness:** we measure error parity — RMSE and MAE — across demand tiers and geography. As the tier numbers showed, accuracy concentrates in high-demand stations, and on the quietest stations R^2 is near zero. We don’t hide that — we *surface* it so operators don’t over-trust a quiet-station forecast.

Drift: we monitor all four types with Evidently — **feature, target, prediction and concept** drift. On the October-2025 data you can see feature drift is detected on a majority of columns, which is exactly the kind of seasonal shift the system is meant to catch. That’s the hook for scheduled monitoring and retraining. That’s the modelling story — over to Rui for what we do with it.”

Hit these:

- Fairness = **error parity** across tiers/geography; weak on low-demand → flagged, not hidden.
- **All four** Evidently drift types (feature / target / prediction / concept), shown on Oct-2025.

Explain it simply — the “12-year-old” version (use this if a judge asks “what *is* drift?” or you want to teach it on the spot):

Big idea: “Drift just asks one question — *has the world changed since we taught the model?* We trained it on 2024; now it’s October 2025. Is that a different world? We check four things.”

- **1. Feature drift — did the *inputs* change?** Put all the 2024 temperatures in one pile and all the Oct-2025 temperatures in another, and measure how far apart the two piles sit. That gap is the **KS number**: 0 means the piles are identical, 1 means totally different. October is a different, colder month, so **month** (KS 0.75) and **temperature** (0.30) clearly moved — **12 of 15** inputs shifted.
- **2. Target drift — did the *answer* change?** The answer is the real number of trips in each 15 minutes. People ride a little differently in autumn, so the thing we’re trying to predict also shifted a bit (KS 0.11 — a small move).
- **3. Prediction drift — did the model’s *guesses* change?** If the inputs move, the model’s outputs naturally move with them (KS 0.36). That’s *expected* — the model is just reacting to a new month, it’s not a bug.
- **4. Concept drift — did the *rule* change?** This is the one that really matters. Even when all the numbers move, the underlying *rule* — “warm weekday evening downtown → lots of trips” — might still be true. We test it by asking: *is the model still as accurate now as it was before?* Its R^2 was 0.327 then and 0.334 now — basically identical — so the rule still holds. **No concept drift.**

Punchline: “The first three just say *the world looks different* — keep watching. Concept drift would say *the model is now actually getting it wrong* — time to retrain. Here only the first three fired and the rule held, so the model is **still trustworthy**; we’d schedule a retrain as a precaution, not an emergency. (And some of that input drift is expected *by design* — we use 2024 as the baseline for every year — so we review it by hand rather than retraining on a blind timer.)”

Reading the KS numbers — “isn’t that just normal variance?”

What KS is. The Kolmogorov–Smirnov statistic compares two distributions by stacking each into a cumulative curve and measuring the *single biggest gap* between the curves. It runs **0 to 1**: 0 = the two look identical, 1 = no overlap. It reacts to *any* change of shape — mean, spread or form — not only the average.

Your instinct is right — two samples always differ a little. Even drawing twice from the *same* distribution gives a small non-zero KS, and that noise *shrinks* as the sample grows. With our $\approx 50,000$ -row samples the pure-noise floor is only about **0.009**, and the test flags *anything* above it as “drift” ($p < 0.05$). So at this size the yes/no flag is almost always “yes”; the honest

read is the **size** of KS, not the flag. You can see the floor directly: the three features that did *not* flag — `slot_sin` 0.005, `slot_cos` / `precipitation` 0.007 — sit right at ≈ 0.005 – 0.009 . *That* is what “just variance” looks like. Rough scale: <0.1 small, 0.1 – 0.2 moderate, >0.2 large, >0.5 huge.

So what really drifted (departure, Oct-2025 vs 2024)?

- **Big & structural — expected:** month 0.75. We score a single October slice against a *full-year* 2024 reference, so the month feature differs *by construction* — a windowing artifact, not real-world drift.
- **Seasonal — real but expected:** temperature 0.30, weather_code 0.21, wind 0.11; and the 2024 historical baselines (`hist_avg_demand` and the recent-lag baselines) ≈ 0.25 — an autumn month genuinely differs from the year-round average. The model is built to handle weather and season.
- **Essentially noise — nothing moved:** dayofweek 0.03, latitude 0.03, longitude 0.06, precipitation / `slot_cos` 0.007, `slot_sin` 0.005. The time-of-day cycle and geography are stable.

Target 0.11 — autumn demand sits a bit lower / differently shaped (seasonal). **Prediction** 0.36 — the model *correctly* maps the shifted inputs to a shifted output range; expected, not a fault.

The point. A different *distribution* is not a broken *model*. The only drift that means “retrain now” is **concept** drift — the input $\rightarrow$ output *rule* changing — and we test that directly with accuracy: R^2 held at $0.327 \rightarrow 0.334$. So our drift is the *expected* seasonal / windowing kind, the learned rule still holds, and the model stays valid. (By design we use 2024 as the baseline for every year, so we review this by hand rather than auto-retraining on a timer.)

Transition: “Over to Rui — turning these forecasts into a rebalancing plan.”

Likely questions (be ready)

- **“Why is R^2 so low?”** — It’s a 15-minute, per-station target: near-Poisson, mostly zeros, huge per-slot variance, so R^2 understates skill. We predict the conditional mean well (MAE ≈ 0.6 trip). It beats the naive baseline, is stable out-of-sample, and reaches R^2 0.39 on the busy stations where rebalancing decisions actually happen. Aggregate it to hourly/station-day and it looks much higher.
- **“Why LightGBM and not the FLAML pick / XGBoost?”** — We didn’t decide; the search did. FLAML and Optuna both explored the space; `lgbm_optuna` had the lowest validation RMSE for *both* targets, so it was auto-selected and promoted.
- **“Isn’t target encoding a leakage risk?”** — Encoders and the 2024 baselines are fit on **train only** (the baselines leave-one-out), then applied forward to May/Oct-2025. Strict temporal split, no peeking. (Sarah’s slide 3.)
- **“What happens when drift fires?”** — Evidently flags it; that’s the trigger for a scheduled re-run of the same pipeline to retrain and re-register a fresh production model — the roadmap item Rui mentions.
- **“Departures and arrivals — two models or one?”** — One shared pipeline, run twice (once per target). Same features, same selection logic; their difference is the rebalancing signal.

Timing: 5 slides in ≈ 2 min is brisk — Slide 9 (Results) is your anchor; if you’re running long, compress Slides 8 and 10 to one sentence each. Keep the naive-baseline and tier- R^2 numbers — they carry the “it’s a real, useful model” argument.